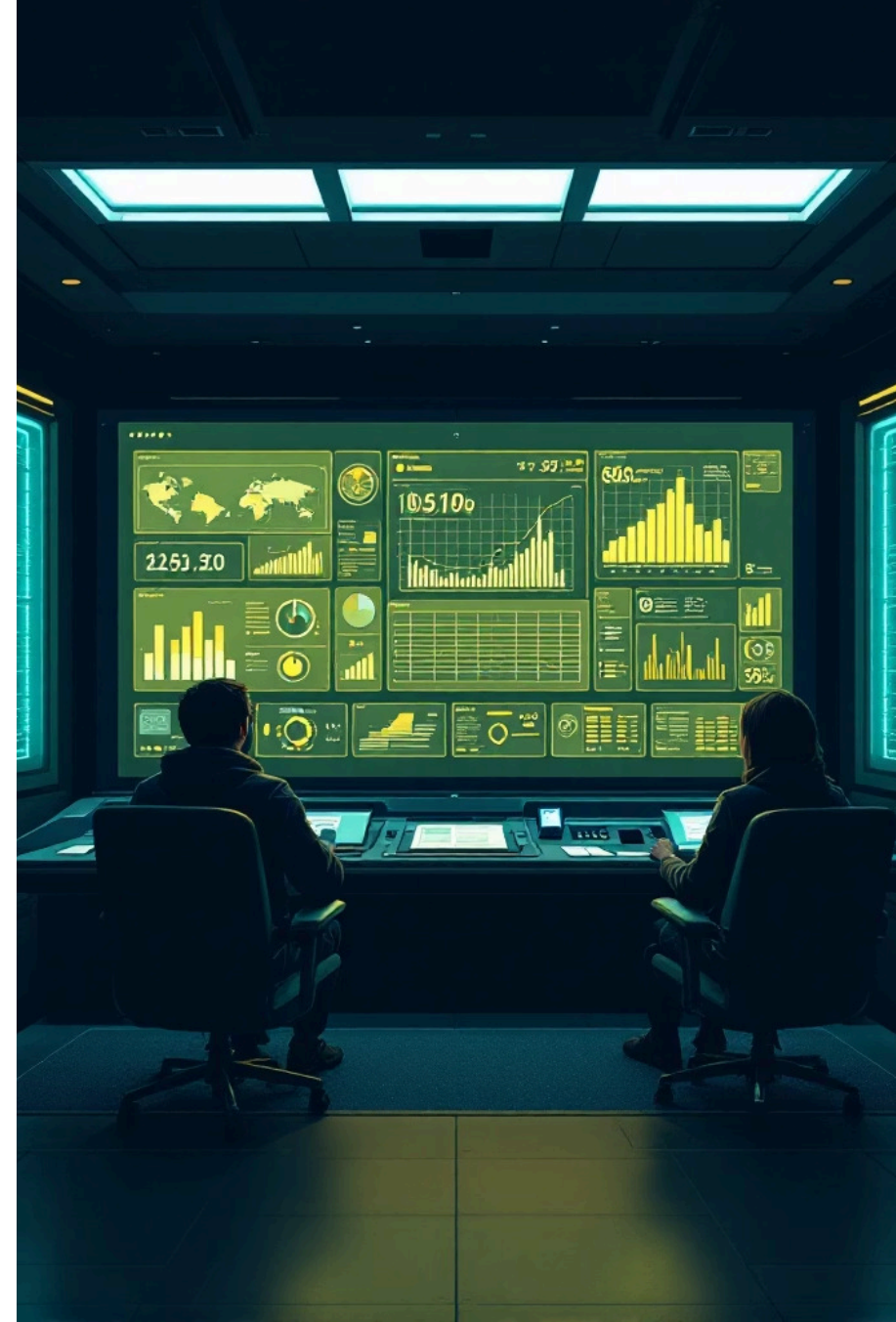


UNLOX

Real-Time Computer Vision Dashboards

From model output to live, interactive applications — building production-ready dashboards with Streamlit and OpenCV



UNLOCK

CHAPTER 19

What We're Building Today

This session bridges the critical gap between model development and real-world application. By the end, you'll have a fully functional, interactive dashboard that processes live video, runs object detection, and triggers real-time alerts — all built in Python.



Streamlit Dashboards

Build interactive web apps for CV models



OpenCV Integration

Capture and process live video streams



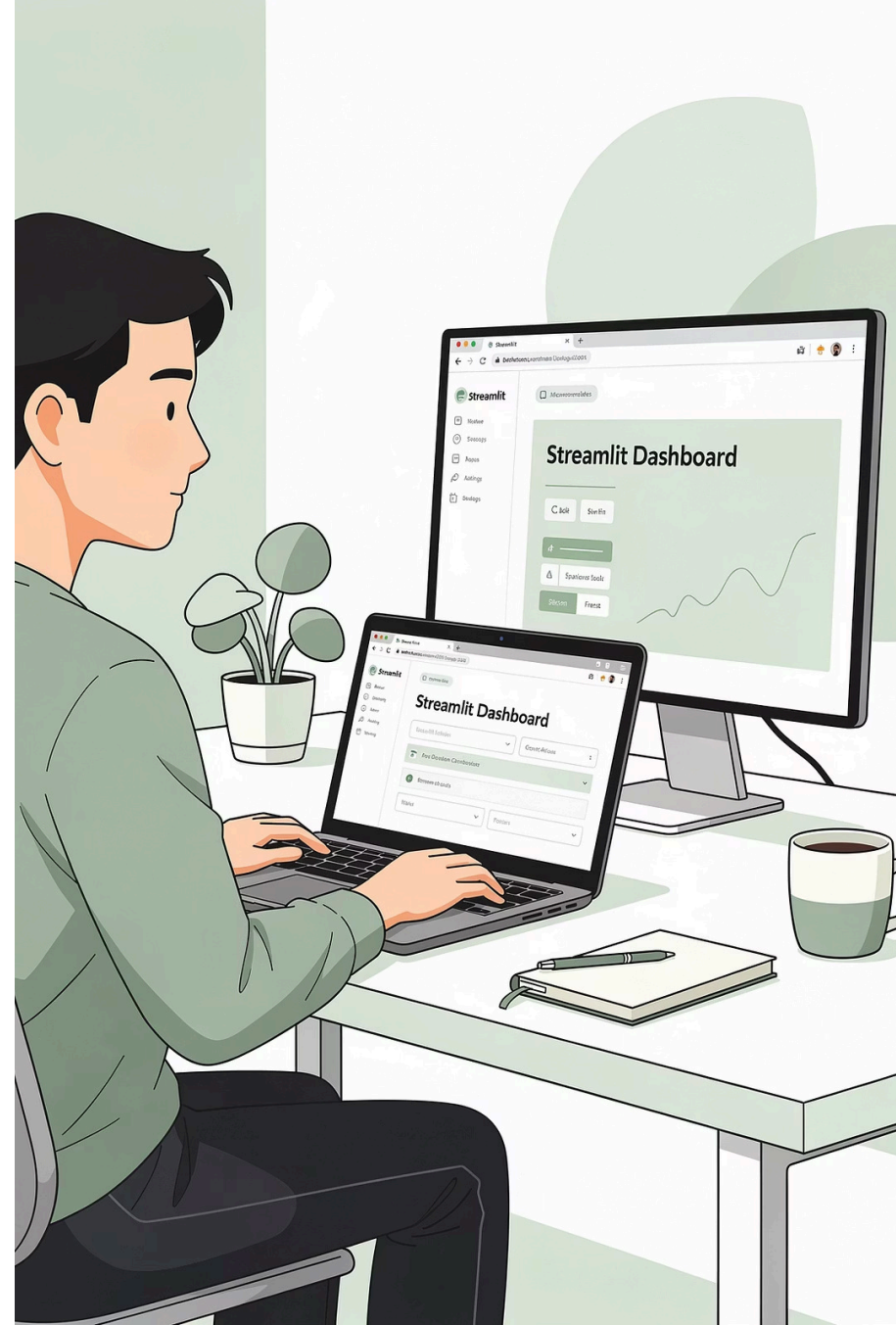
Real-Time Alerts

Trigger notifications on detected events

UNLOX SECTION 19.1

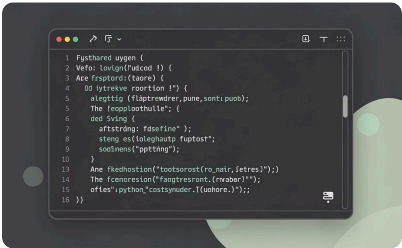
Creating Real-Time Dashboards

Turning your computer vision models into interactive, shareable web applications with Streamlit



Why Streamlit for Computer Vision?

Streamlit is an open-source Python library purpose-built for data science and machine learning applications. It lets you transform Python scripts into polished, interactive web apps with minimal frontend code — no HTML, CSS, or JavaScript required. For CV practitioners, this means you can go from a working model to a deployable dashboard in minutes, not days.



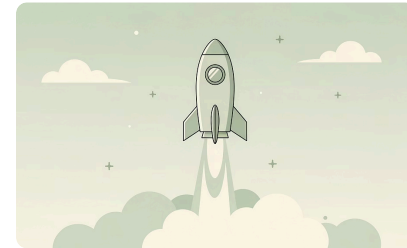
Pure Python

Write your entire dashboard in Python. No frontend frameworks, no context switching — just the language you already know.



Built-In Widgets

Sliders, file uploaders, camera inputs, and selectboxes are available out of the box for rapid prototyping.



Instant Deployment

Share your app via Streamlit Cloud or deploy it on your own infrastructure with a single command.

UNLOCK

Setting Up Your First Streamlit App

Getting started is straightforward. Install Streamlit via pip, create a Python file, and launch it from your terminal. The framework watches your file for changes and hot-reloads the browser automatically — a massive productivity boost during development.

Quick Start Commands

```
# Install Streamlit
pip install streamlit

# Create your app file
touch app.py

# Run the dashboard
streamlit run app.py
```

Your app will launch at `localhost:8501` with live reloading enabled by default.

Key Dependencies

- **streamlit** — dashboard framework
- **opencv-python** — video capture & processing
- **numpy** — array manipulation
- **Pillow** — image format handling
- **torch / tensorflow** — your model backend

Core Streamlit Components for CV

Streamlit offers several components that are particularly powerful for computer vision workflows. Understanding these core building blocks will let you compose sophisticated dashboards that handle everything from image uploads to real-time video rendering.

1

`st.image()`

Display images and NumPy arrays directly. Supports OpenCV BGR arrays, PIL images, and file paths. Add captions and control width dynamically.

2

`st.camera_input()`

Capture snapshots from the user's webcam directly in the browser. Returns a file-like object you can decode with OpenCV or PIL.

3

`st.file_uploader()`

Accept image or video uploads from users. Filter by file type with the `type` parameter — e.g., `["jpg", "png", "mp4"]`.

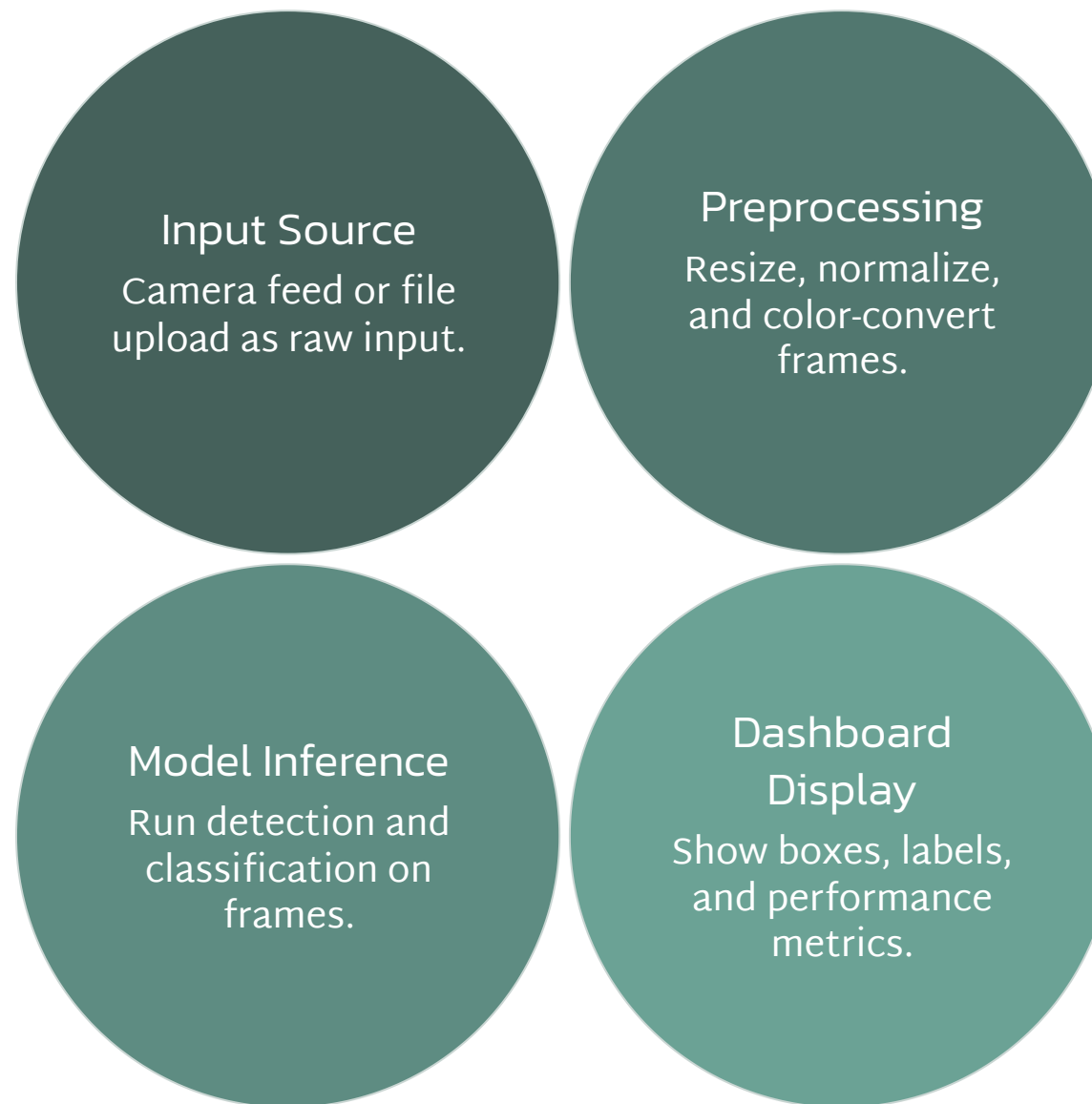
4

`st.empty()`

Create a placeholder that can be updated in a loop — essential for rendering live video frames at high refresh rates.

Dashboard Architecture Pattern

A well-structured Streamlit CV dashboard follows a clear data flow: input → preprocessing → inference → visualization. This pattern keeps your code modular and makes it easy to swap models, adjust thresholds, or add new features without rewriting the entire app.



Each stage is a separate function in your codebase. The Streamlit event loop calls them sequentially on every rerun, keeping the UI in sync with your model's output. Use `st.cache_resource` to load your model only once — not on every frame.

UNLOCK SECTION 19.2

OpenCV & Real-Time Alerts

Integrating live video capture, object detection overlays, and event-driven notifications into your dashboard



Capturing Live Video with OpenCV

OpenCV (Open Source Computer Vision Library) is the industry-standard library for computer vision tasks. Its `VideoCapture` class provides a unified interface for reading from webcams, IP cameras, RTSP streams, and video files. Integrating it with Streamlit requires a frame-by-frame rendering loop using `st.empty()` as a dynamic placeholder.

Live Capture Loop

```
import cv2
import streamlit as st

cap = cv2.VideoCapture(0)
frame_holder = st.empty()

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
    # Convert BGR to RGB for Streamlit
    frame_rgb = cv2.cvtColor(
        frame, cv2.COLOR_BGR2RGB
    )
    frame_holder.image(frame_rgb)

cap.release()
```

Key Concepts

- `VideoCapture(0)`

Index 0 grabs the default webcam. Pass a URL string for IP cameras or RTSP streams.

- `BGR → RGB`

OpenCV reads frames in BGR format. Always convert to RGB before passing to Streamlit's `st.image()`.

- `Frame Placeholder`

`st.empty()` updates in-place, avoiding the creation of a new image element on each iteration.

UNLOCK

Drawing Bounding Boxes & Labels

Once your object detection model returns predictions, you need to overlay bounding boxes and class labels onto each frame before displaying it. OpenCV provides `cv2.rectangle()` and `cv2.putText()` for drawing directly onto NumPy arrays — making this both fast and memory-efficient.



Bounding Boxes

Use `cv2.rectangle(frame, (x1, y1), (x2, y2), color, thickness)` to draw boxes around detected objects. Color-code by class for visual clarity.



Class Labels & Confidence

Render the class name and confidence score above each box with `cv2.putText()`. Format as "person 0.94" for quick readability.



Sidebar Metrics

Use `st.sidebar.metric()` to display FPS, total object count, and per-class breakdowns alongside the video feed.

UNLOX

Implementing Real-Time Alerts

Real-time alerts transform a passive monitoring dashboard into an active response system. You define trigger conditions — such as detecting a specific object class, exceeding a count threshold, or an object entering a restricted zone — and the system fires notifications instantly. This is where your CV models start delivering real operational value.

Class-Based Triggers

Fire an alert when a target class (e.g., "fire", "weapon") appears in the frame with confidence above your threshold.

Count Thresholds

Trigger when the number of detected objects exceeds a limit — useful for crowd monitoring or inventory tracking.

Zone Intrusion

Define polygonal regions of interest. Alert when a detected object's centroid enters a restricted area.



Alert Logic Pattern

The alert system sits inside your frame loop, evaluating detections against your configured rules after every inference pass. Use a cooldown timer to prevent alert flooding — you don't want hundreds of duplicate notifications for a single event. Here's the core pattern:

```
import time

COOLDOWN = 10 # seconds
last_alert = 0

for det in detections:
    if det.label == "person" \
    and det.confidence > 0.8:
        now = time.time()
        if now - last_alert > COOLDOWN:
            send_alert(
                f"Person detected: "
                f"{det.confidence:.0%}"
            )
            st.warning("Person detected")
```

UNLOCK

Alert Delivery Channels

Your alert system is only as useful as its delivery mechanism. Choose channels based on urgency, audience, and infrastructure. Most production systems use a combination — for example, an in-dashboard banner for operators plus a Slack message for the engineering team.



In-Dashboard Banners

Use `st.warning()`, `st.error()`, or `st.toast()` for immediate visual feedback. Best for live operators watching the dashboard in real time.



Email & SMS

Integrate with `smtplib` or Twilio for asynchronous notifications. Include a frame snapshot as an attachment for context.



Slack / Webhook

POST to a Slack webhook or custom endpoint for team-wide visibility. Structure the payload as JSON with detection metadata.



Event Logging

Write every alert to a database or CSV log with timestamp, class, confidence, and bounding box coordinates for audit trails.

Performance Optimization Tips

Real-time video processing is computationally intensive. Without optimization, your dashboard will stutter, drop frames, or become unresponsive. These techniques keep your pipeline fast and your UI smooth, even on modest hardware.

Cache Your Model

Use `@st.cache_resource` to load the model once and persist it across reruns. This avoids reloading weights on every frame — a common beginner mistake that destroys performance.

Skip Frames Strategically

Run inference every 2nd or 3rd frame instead of every frame. Display all frames but only draw new detections when inference completes, maintaining visual smoothness.

Resize Before Inference

Downscale frames to your model's expected input size (e.g., 640×640) before passing them in. Processing full-resolution 1080p frames wastes cycles.

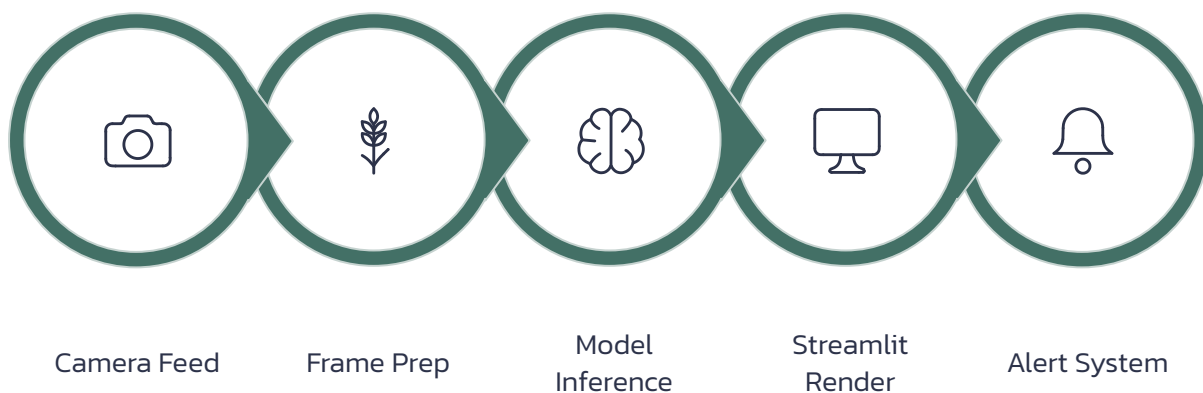
Use Threading for Capture

Run `cv2.VideoCapture.read()` in a background thread to decouple I/O-bound frame capture from CPU-bound inference and rendering.

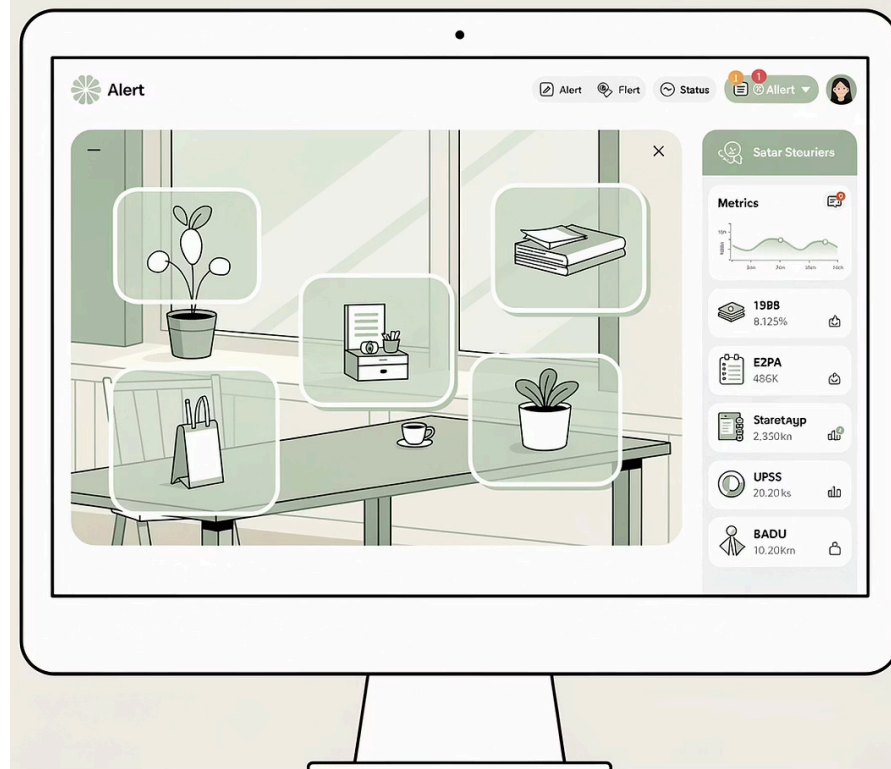
UNLOX

Putting It All Together

Your complete real-time CV dashboard combines every component we've covered into a single, cohesive application. Here's how the full architecture flows from camera to alert.



With this architecture in place, you have a production-capable system that can monitor any video source, detect objects of interest, visualize results in real time, and notify stakeholders when critical events occur — all built with Python and open-source tools.



UNLOCK

Key Takeaways

Streamlit = Rapid Prototyping

Build interactive CV dashboards in pure Python with hot-reloading, built-in widgets, and one-command deployment.

OpenCV = Video Pipeline

Use `VideoCapture` for any source, convert BGR to RGB, and render frames via `st.empty()` for smooth playback.

Alerts = Actionable Intelligence

Define trigger rules, apply cooldowns to prevent spam, and dispatch through multiple channels for operational impact.

Next up: We'll deploy this dashboard to the cloud with Docker and set up persistent alert logging with a real database backend.